

RESEARCH PAPER REVIEW: DEEP RESIDUAL LEARNING FOR IMAGE RECOGNITION

Paper Reference: *Deep Residual Learning for Image Recognition* (Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun; CVPR 2016)

Name: Natan Amanuel Mamo

Introduction

Deep learning has significantly advanced the field of computer vision, particularly through Convolutional Neural Networks (CNNs). However, increasing the depth of CNNs introduced a major challenge: deeper networks became increasingly difficult to train. The paper *Deep Residual Learning for Image Recognition* introduced the concept of residual learning, enabling the successful training of extremely deep neural networks through shortcut connections.

The proposed architecture, ResNet (Residual Network), achieved state-of-the-art performance on ImageNet and won the ILSVRC 2015 classification competition with a top-5 error rate of 3.57%. Today, residual connections are widely used across modern deep learning architectures, including object detection models, segmentation networks, and Transformers.

1. What problem were the authors trying to solve?

The authors sought to address the difficulties of training deep neural networks as their depth scales. Network depth is of crucial importance for extracting rich, hierarchical feature representations. However, as network depth is increased to capture complex features, convergence and optimization become extremely difficult. The main focus was to overcome the **degradation problem**, where adding more layers to a model leads to a rapid drop in accuracy and a higher training error, preventing extremely deep architectures from realizing their theoretical capacity.

2. Why does simply making CNNs deeper eventually stop improving performance?

Theoretically, a deeper model should have a solution space that contains its shallower counterpart as a subspace. If the added layers are constructed as simple identity mappings, the deeper network should perform at least as well as the shallower one.

However, in practice, as network depth increases, optimization solvers (like Stochastic Gradient Descent with backpropagation) struggle to find these identity mappings using stacks of nonlinear layers. This leads to the **degradation problem**, where accuracy saturates and then degrades rapidly. Stacking standard layers introduces optimization complexities and low convergence rates, which prevent solvers from reducing training error in feasible timeframes.

3. Explain the difference between: Vanishing Gradients, Overfitting, and The Degradation Problem

- **Vanishing (or Exploding) Gradients:** This is an optimization hurdle where gradients shrink (or grow) exponentially as they propagate backward through many layers. When gradients vanish, weight updates in the early layers become negligible, preventing the network from converging. This issue has been largely mitigated through normalized initializations and intermediate normalization layers (like Batch Normalization).
- **Overfitting:** Overfitting occurs when a model fits the training dataset *too well*, capturing noise alongside the underlying patterns. It is characterized by **very low training error** but **high validation/testing error**.
- **The Degradation Problem:** Unlike overfitting, degradation is an optimization failure rather than generalization failure. It is marked by a **higher training error** (and consequently higher test error) as the network becomes deeper. Because both training and test errors increase concurrently, it proves that the optimization solver is unable to properly fit the deep model, despite having the structural capacity to do so.

4. What is Residual Learning?

Instead of forcing stacked nonlinear layers to directly approximate a desired target mapping $H(x)$, the residual learning framework reformulates the layers to approximate a **residual mapping** $F(x) := H(x) - x$.

The original, target mapping is recast as:

$$H(x) = F(x) + x$$

This approach assumes that optimizing a residual mapping is significantly easier than optimizing the original, unreferenced mapping. Under this framework, if an identity mapping is optimal, the solver can easily push the weights of the nonlinear layers toward zero. This acts as a form of preconditioning, allowing information to flow through the network via identity shortcuts.

5. Explain the residual equation in your own words

The fundamental residual block is defined by the following equation:

$$y = F(x, \{W_i\}) + x$$

Where:

- x represents the input vector to the residual block.
- $F(x, \{W_i\})$ represents the residual mapping function learned by the stacked weight layers (typically a set of convolutional and activation layers).
- y is the output vector of the residual block.

In plain terms: Instead of forcing the layers to learn a completely new, transformed version of the input, we configure them to only learn the *changes* (the residual) needed to adjust the input. We then add the original input x directly back to this calculated adjustment via an identity shortcut. The final activation (ReLU) is then applied to the sum y .

6. How do shortcut (skip) connections work?

Shortcut connections are feedforward pathways that bypass one or more layers. In ResNet, the shortcut connections are strictly **identity mappings**. They carry the input x forward without

adding any learnable parameters or computational complexity, and their outputs are simply added element-wise to the output of the bypassed convolutional layers. Because they do not introduce parameters, they allow a fair comparison with plain CNNs.

7. What is the difference between:

A. Plain CNN Blocks vs. Residual Blocks

- **Plain CNN Blocks:** Stack convolution, batch normalization, and activation layers sequentially. The signal must pass through every transformation, making it highly susceptible to optimization bottlenecks in deep architectures.
- **Residual Blocks:** Include a parallel skip pathway that adds the original input x to the processed output $F(x)$. This structure provides a direct highway for both forward activations and backward-flowing gradients.

B. Identity Shortcuts vs. Projection Shortcuts

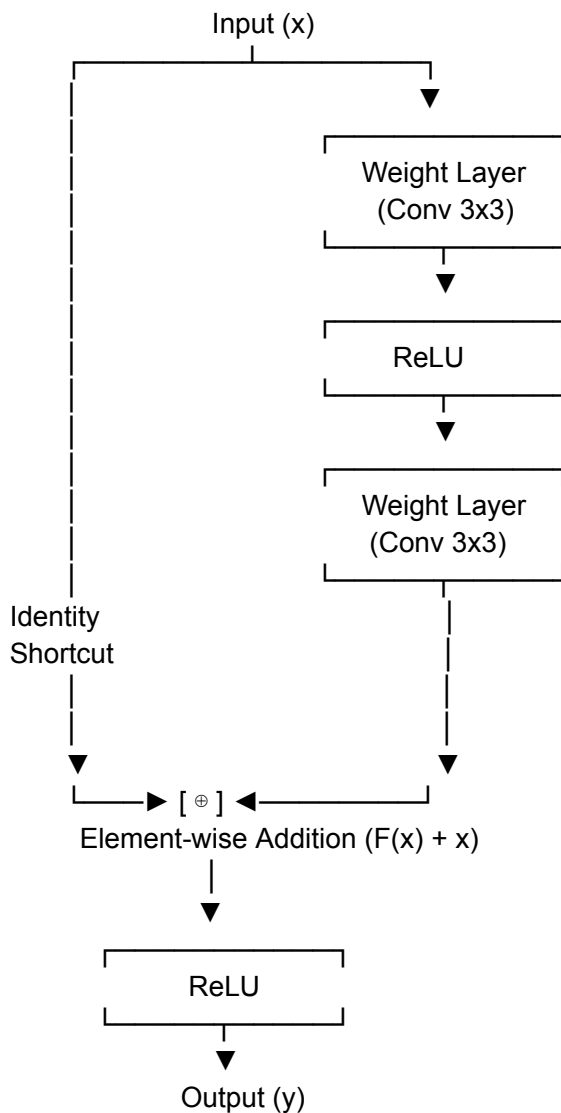
- **Identity Shortcuts:** Perform a direct routing of the input x without modification ($y = F(x) + x$). They require zero parameters and zero computational overhead. They can only be used when the dimensions of $F(x)$ and x are identical.
- **Projection Shortcuts:** Used when spatial dimensions or channel sizes change (e.g., when the feature map is halved and channels are doubled). They apply a linear projection W_s to match dimensions: $y = F(x) + W_s x$. This is typically implemented via 1×1 convolutions with a stride of 2.

C. Basic Blocks vs. Bottleneck Blocks

- **Basic Blocks:** Typically consist of two sequential 3×3 convolutional layers (as seen in ResNet-18 and ResNet-34).
- **Bottleneck Blocks:** Designed to reduce computational cost for deeper networks (ResNet-50, 101, 152). They stack three layers: a 1×1 convolution (to reduce/squeeze dimensions), a 3×3 convolution, and a final 1×1 convolution (to restore/expand dimensions). This structure drastically limits the number of parameters while maintaining identical complexity.

Original Diagram: Residual Block Architecture

The diagram below illustrates a standard **Basic Residual Block** with an identity shortcut connection.



8. Summary of Experiments Performed in the Paper

The authors evaluated their models on two main datasets: **ImageNet 2012** and **CIFAR-10**.

- **ImageNet Experiments:**

- They compared 18-layer and 34-layer **plain** networks against 18-layer and 34-layer **residual** networks (ResNets).
- They implemented deeper architectures (50, 101, and 152 layers) using **bottleneck blocks** to keep computational complexity low.
- They evaluated different shortcut configurations: Option A (zero-padding), Option B (projection only for increasing dimensions), and Option C (all projection shortcuts).

- **CIFAR-10 Experiments:**
 - They tested deep ResNet variants starting from 20 layers up to 110 layers, and even explored an aggressively deep model of **1,202 layers**.
 - They analyzed the standard deviations of layer responses to examine the impact of residual mappings.

9. What evidence convinced you that the proposed method actually works?

Several key experimental results in the paper prove the effectiveness of ResNets:

1. **Reversal of the Degradation Trend (ImageNet Validation):** As shown in **Figure 4 (left)**, the 34-layer *plain* network exhibited a higher training and validation error than the 18-layer plain network. However, in **Figure 4 (right)**, the 34-layer *ResNet* successfully achieved a **3.5% lower top-1 error** than the 18-layer ResNet, showing that training residual mappings allows accuracy to scale with depth.
2. **Ultra-Deep Performance:** The 152-layer ResNet reached a remarkable single-model top-5 validation error of **4.49%**, outperforming all previous complex ensemble models. The ResNet ensemble achieved a **3.57% top-5 error on the ImageNet test set**, winning first place in the ILSVRC 2015 classification task.
3. **CIFAR-10 Depth Scaling:** Extremely deep networks (110-layer ResNet) converged without optimization issues and achieved state-of-the-art accuracy.
4. **Smaller Layer Responses:** Standard deviation plots of layer responses (**Figure 7**) show that ResNet layers have smaller responses than plain counterparts, proving that residual functions are indeed closer to zero and easier to optimize.

10. What limitations or open questions remain after reading the paper?

- **Extreme Overfitting at Over-Extreme Depth:** While the 1,202-layer ResNet was optimized with zero difficulty (training error <0.1%), it ultimately achieved a higher test error (7.93%) than the 110-layer model (6.43%). The paper identifies this as overfitting due to the model being unnecessarily large for CIFAR-10, leaving the question of how to properly regularize or utilize such colossal architectures open.
- **Theoretical Mechanism of Residual Optimization:** The hypothesis that optimizing a residual function is fundamentally easier than the original mapping is confirmed empirically, but a rigorous, formal mathematical proof of why the optimization landscapes are so well-behaved remains open.

Short Reflection

The most surprising aspect of this paper is the simplicity of the solution. The introduction of identity shortcut connections does not require any additional math layers, complex gating mechanisms (unlike Highway Networks), or extra parameters. Instead, it purely changes how the training problem is preconditioned. It is fascinating how a simple modification—adding the input tensor back to the output—unlocked the ability to train networks ten times deeper than before, showing that structural formulation often matters just as much as raw optimization algorithms.